BLOOD DONOR REGISTRY AND EMERGENCY REQUEST SYSTEM USING ZOHO CREATOR

A PROJECT REPORT

Submitted by

S K ADISHWAR SEELAN

(2116241401004)

in partial fulfillment of the End Semester Examination for the course

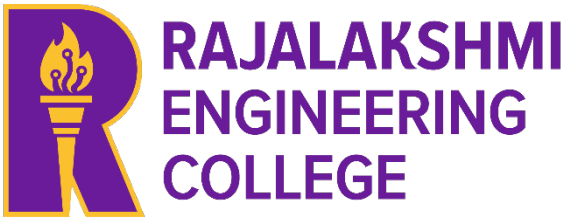
CB23F35

LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF TECHNOLOGY

COMPUTER SCIENCE AND BUSINESS SYSTEMS



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

NOVEMBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled “Blood Donor Registry and Emergency Request System using Zoho Creator” is the bonafide work of S.K Adishwar Seelan (2116241401004), who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year 2026–2027, Semester V, under my supervision.

SIGNATURE

Dr. K.R Sowmia,
HEAD OF THE DEPARTMENT
Department of Computer Science and Business Systems
Rajalakshmi Engineering College
Thandalam – 602 105

SIGNATURE

Mr. M.L Alphin Ezhil Manuel
Assistant Professor (SG)
Department of Computer Science and Business Systems
Rajalakshmi Engineering College
Thandalam – 602 105

Submitted for the End Semester Examination (Individual Project) held in November 2026.

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I, S.K Adishwar Seelan (2116241401004), hereby declare that the project report titled “Blood Donor Registry and Emergency Request System” using Zoho Creator, submitted for the End Semester Examination of the course CB23F35 – Low-Code Application Development using Zoho Creator, is a record of original work carried out by me individually. The Zoho Creator application described in this report — including its data model, forms, reports, pages, Deluge scripts, workflows and integrations — was designed and developed by me.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. Every template, sample application, AI-assisted feature (such as Zia or the AI application builder), code snippet or external resource used in this work has been duly acknowledged in the report.

SIGNATURE OF THE STUDENT

Date: 03.11.2026 S.K ADISHWAR SEELAN

ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman Mr. S. Meganathan, B.E., F.I.E., Chairperson Dr. Thangam Meganathan, Ph.D., and Vice Chairman , Mr. Abhay Shankar Meganathan, B.E., M.S., for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal, Dr. S.N. Murugesan, M.E., Ph.D., Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank Dr. K.R. Sowmia, Ph.D., Head of the Department, Department of Computer Science and Business Systems, for the guidance and support provided throughout the course.

I owe my sincere thanks to my course faculty, Mr. M.L Alphin Ezhil Manuel, Assistant Professor (SG), Department of Computer Science and Business Systems for the valuable guidance, reviews and feedback that shaped this application and report.

I also acknowledge Zoho Corporation for providing access to the Zoho Creator platform and learning resources, and I thank my family and friends for their unwavering support.

S K ADISHWAR SEELAN

ABSTRACT

Blood banks, hospitals and student volunteer groups in Chennai still match patients with donors through WhatsApp forwards, paper registers and personal phone calls. Volunteers cannot see which donors are medically eligible, because the date of each donor's last donation is rarely recorded, and an urgent request can take hours to fill while the same few donors are called repeatedly. There is no single record of open requests, no audit of who donated, and no way to report on donor availability by blood group. This project builds the Blood Donor Registry and Emergency Request System on the Zoho Creator low-code platform. Three related forms — Donor Registration, Blood Request and Donation Log — capture donors, patient requests and completed donations, linked through lookup fields. Two short Deluge scripts validate donor age, compute eligibility under the 90-day deferral rule, find eligible donors whose blood group is medically compatible with the requested group, and email each of them automatically. Coordinators close a request by moving it to Fulfilled on a Kanban board. List, Kanban and Calendar reports and a UI-Builder dashboard give coordinators a live view of donors and requests, and permission profiles separate administrators from coordinators. Every workflow was unit-tested in the Development environment with valid and invalid records; the defects found during development were fixed. Matching a request to compatible, eligible donors now happens in a single submission instead of a chain of manual phone calls, and the status of every request is visible at a glance. The application is prepared for release through the Development → Stage → Production environments.

Keywords: ***Low-Code Development; Zoho Creator; Deluge Scripting; Workflow Automation; Role-Based Access Control; Blood Donor Management***

PROJECT AT A GLANCE

Attribute	Details
Project Title	Blood Donor Registry and Emergency Request System using Zoho Creator
Application Domain	Healthcare – voluntary blood donation and emergency blood requests
Student Name & Register No.	S.K Adishwar Seelan – 2116241401004
Zoho Creator Edition	Zoho Creator (trial / education workspace)
Application Name (Link Name)	Blood Donor Registry & Emergency Request System (blood-donor-registry-emergency-request-system)
Published / Live URL	https://creatorapp.zoho.in/adishwarseelan.sk.2024.cs215/blood-donor-registry-emergency-request-system/
Demo Credentials (per role)	Admin: application owner account | Coordinator: adishwarseelan.sk.2024.csbs+sherwin@rajalakshmi.edu.in (Write profile) | Viewer / Donor: lalitha – adishwarseelan.sk.2024.csbs+lalitha@rajalakshmi.edu.in (Read profile)
Number of Forms / Subforms	3 / 0
Number of Reports (by type)	List: 3 Kanban: 1 Calendar: 1 Chart (on page): 1 Pivot: 0
Number of Pages / Dashboards	1 (Donor Dashboard)
Number of Workflows	Form: 2 Schedule: 0 Approval: 0 Blueprint: 0 Batch: 0
Deluge Functions / Custom Actions	0 functions / 0 custom actions (2 workflow scripts)
Integrations	Zoho Mail (sendmail); Creator REST API v2 (test data)
Roles & Profiles Defined	Administrator (owner); Write profile – Blood Bank Coordinator; Read profile – Viewer / Donor
Environments Used	Development → Stage → Production

TABLE OF CONTENTS

ABSTRACT....iv
PROJECT AT A GLANCE....v
LIST OF TABLES....viii
LIST OF FIGURES....ix
LIST OF ABBREVIATIONS....x
INTRODUCTION....1
1.1 Background....1
1.2 Problem Statement....1
1.3 Objectives....2
1.4 Scope of the Project....2
1.5 Justification for a Low-Code Approach....3
1.6 Organisation of the Report....3
LITERATURE SURVEY AND SYSTEM STUDY....4
2.1 Study of the Existing System....4
2.2 Limitations of the Existing System....5
2.3 Low-Code versus Traditional Development....5
2.4 Literature Survey....5
2.5 Emerging Trends in Low-Code Development....6
2.6 Proposed System....6
REQUIREMENT ANALYSIS....7
3.1 Stakeholders and User Roles....7
3.2 Functional Requirements....7
3.3 Non-Functional Requirements....7
3.4 Hardware and Software Requirements....8
3.5 Use Case Model....8
SYSTEM DESIGN....9
4.1 System Architecture....9
4.2 Data Model....9
4.3 Form Design....10
4.4 Process and Workflow Design....10
4.5 User Interface and Navigation Design....11
4.6 Access Control Design....11
IMPLEMENTATION....12
5.1 Application Setup....12
5.2 Forms and Data Capture....12
5.3 Reports....14
5.4 Pages and Dashboards....15
5.5 Deluge Scripting....16
5.6 Workflows and Automation....18
5.7 Integrations....19
5.8 User Management and Role-Based Access....19
5.9 AI and Analytics Features....19
5.10 Pro-Code Extension....20
5.11 Mobile and Portal Access....20
TESTING AND DEBUGGING....21
6.1 Testing Strategy....21
6.2 Test Cases....21

6.3 Debugging Deluge Scripts....21
6.4 Defect Log....21
6.5 User Acceptance Testing....22
DEPLOYMENT, SECURITY AND GOVERNANCE....23
7.1 Environments and Deployment Strategy....23
7.2 Performance Optimisation....23
7.3 Security Features....23
7.4 Governance and Administration....24
RESULTS AND DISCUSSION....25
8.1 Application Output....25
8.2 Achievement of Objectives....26
8.3 Comparison with the Existing System....26
8.4 Limitations of the Low-Code Approach....26
8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum....26
CONCLUSION AND FUTURE ENHANCEMENTS....27
9.1 Conclusion....27
9.2 Future Enhancements....27
9.3 Learning Outcomes and Career Relevance....27
REFERENCES....28
APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST....29
APPENDIX B – COURSE OUTCOME MAPPING....31
APPENDIX C – COMPLETE DELUGE SOURCE CODE....32
APPENDIX D – USER MANUAL....34
D.1 Accessing the Application....34
D.2 Instructions for Blood Bank Coordinator....34
D.3 Instructions for Administrator....34

LIST OF TABLES

Table 6.1: Test Cases.....21
Table A.1: Feature Coverage Checklist.....29
Table B.1: Mapping of Course Outcomes to Report Chapters.....31

LIST OF FIGURES

Figure 2.1: As-Is Process Flow of the Existing Manual Donor-Matching Process.....4
Figure 3.1: Use Case Diagram of the Blood Donor Registry.....8
Figure 4.1: System Architecture of the Blood Donor Registry.....9
Figure 4.2: Entity–Relationship Diagram of the Application.....9
Figure 4.3: Process Flow for Blood Request Matching and Donation.....10
Figure 4.4: Navigation Structure and Wireframe of the Donor Dashboard.....11
Figure 5.1: Application Home (Donor Dashboard) and Navigation Sections.....12
Figure 5.2: Donor Registration Form in the Form Builder.....13
Figure 5.3: Donor Registration Form in Live Mode.....13
Figure 5.4: Blood Request Form in Live Mode.....13
Figure 5.5: Donation Log Form with Lookup Fields to Blood Request and Donor….14
Figure 5.6: All Donors Report Grouped by Blood Group with Quick Filters.....14
Figure 5.7: Blood Requests Report Grouped by Urgency and Sorted by Date Required 15
Figure 5.8: Request Pipeline – Kanban Report by Status.....15
Figure 5.9: Requirement Calendar Report by Date Required.....15
Figure 5.10: Donor Dashboard Page – Navigation Buttons and KPI Panel.....16
Figure 5.11: DS-01 in the Deluge Editor (Donor Registration – On Validate).....18
Figure 5.12: DS-02 in the Deluge Editor (Blood Request – On Success).....18
Figure 5.13: Form Workflows Configured on the Donor Registration Form.....18
Figure 5.14: Form Workflow Configured on the Blood Request Form.....19
Figure 5.15: Permission Profiles under Settings → User Permissions.....19
Figure 5.16: Analytics – Bar Chart of Donors by Blood Group on the Donor Dashboard.....20
Figure 7.1: Environments – Publish to Stage and Production.....23
Figure 7.2: Audit Trail / Administration Settings.....24
Figure 8.1: Scenario Step 1 – Donors Registered and Grouped by Blood Group.....25
Figure 8.2: Scenario Step 2 – Urgent A+ve Request Logged for Ramesh K.....25
Figure 8.3: Scenario Step 3 – Donation Logged and Request Moved to Fulfilled.....26

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
DS	Deluge Script (identifier used in this report)
ER	Entity–Relationship
FR	Functional Requirement
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
LCAP	Low-Code Application Platform
NFR	Non-Functional Requirement
RBAC	Role-Based Access Control
REST	Representational State Transfer
SDLC	Software Development Life Cycle
TC	Test Case
UAT	User Acceptance Testing
UI	User Interface
Zia	Zoho’s AI assistant

CHAPTER 1
INTRODUCTION

1.1 Background

Voluntary blood donation is the main source of blood for hospitals in India. In Chennai, a large share of emergency requests — accident victims, post-operative bleeding, thalassaemia transfusions and maternity complications — are not filled from a blood bank's stock but by finding a willing donor of the right group within a few hours. Much of this matching is done by volunteer networks: college clubs, NSS units and community groups that keep their own donor lists.

A typical volunteer group keeps donor details in a spreadsheet or a paper register and receives requests through WhatsApp messages and phone calls from patients' attenders. A coordinator then searches the list for the requested blood group and phones donors one by one, asking each whether they have donated recently. A donor may donate whole blood only once in roughly every three months, so a donor who gave blood last month must be skipped, but this information is usually not written down.

This project models such a volunteer blood-donor network operating around a college campus in Chennai. The organisation has three groups of people: an administrator who manages the application, blood-bank coordinators who register donors, log incoming requests and record donations, and donors who are contacted when a compatible request arrives. The business process addressed is the complete request lifecycle: registration of donors, capture of a request, identification and notification of eligible compatible donors, recording of the donation and closure of the request.

1.2 Problem Statement

Emergency blood requests are matched to donors manually through phone calls and messaging groups, using donor lists that do not record eligibility or blood-group compatibility, so requests take hours to fill, eligible donors are missed, recently deferred donors are called again, and nobody has a reliable view of which requests are still open.

- Donor data is scattered across spreadsheets and chat histories, with duplicate entries and no standard blood-group format.

- The date of each donor's last donation is not tracked, so coordinators cannot tell who is eligible under the 90-day deferral rule.
- Coordinators search only for an exact blood-group match and overlook compatible donors (for example O-ve donors for any recipient).
- Request status is not recorded, so fulfilled requests keep circulating and overdue ones are not followed up.
- There is no dashboard or report on donor availability by blood group, and no access control over donors' personal data.

1.3 Objectives

- To design a normalised data model of three related forms (donor, request, donation) with field-level validation for donor registration and request capture.
- To automate eligibility checking, compatible-donor matching and donor notification with two short Deluge scripts.
- To provide coordinators with list, Kanban and Calendar reports and a dashboard of donors by blood group.
- To notify donors automatically by email through Zoho Mail.
- To prepare the application for secure release through the Development, Stage and Production environments with permission profiles and governance controls.

1.4 Scope of the Project

In scope: donor registration with age and donation-date checks; automatic eligibility; blood requests with urgency and status; email to eligible donors whose blood group matches; a donation log; list, Kanban and Calendar reports; a dashboard; and permission profiles for administrator, coordinator and read-only users. The app also works in the Zoho Creator mobile app with no extra setup.

Out of scope: blood-bank inventory and component separation, medical screening of donors, integration with the national e-RaktKosh portal, SMS or WhatsApp messaging, payments, and a public customer portal for patients. These are listed as future enhancements in Chapter 9.

1.5 Justification for a Low-Code Approach

The problem is a data-and-workflow problem rather than an algorithmic one: a handful of related records, validation rules, notifications and status changes. Low-code platforms are designed for exactly this class of business application and let a single developer deliver a working system in days rather than months [1], [5]. Zoho Creator provides hosted forms, reports, pages, a relational data store, role-based permissions, environments for staged release and a mobile app out of the box [2], while Deluge scripting covers the custom logic — eligibility, compatibility and notification — that drag-and-drop configuration alone cannot express [3]. Because the volunteer organisation has no IT staff or server budget, the absence of infrastructure to manage, the low cost and the ability for a citizen developer to maintain the application are decisive advantages over a traditional web stack.

1.6 Organisation of the Report

Chapter 2 reviews the existing system and related low-code literature. Chapter 3 presents the requirement analysis. Chapter 4 describes the system design. Chapter 5 details the implementation on Zoho Creator. Chapter 6 covers testing and debugging. Chapter 7 discusses deployment, performance optimisation, security and governance. Chapter 8 presents results and discussion, and Chapter 9 concludes the report with future enhancements.

CHAPTER 2
LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

In the existing system, a patient's attender or a hospital staff member posts a request for blood in WhatsApp groups or calls a known volunteer. The volunteer opens a shared spreadsheet or paper register, filters it by the requested blood group and starts calling donors. Because the last donation date is not recorded, each donor must be asked whether they have donated recently; deferred donors are skipped and the next number is tried. When a donor agrees, they travel to the hospital and donate. The donation is rarely recorded, and the request is closed only informally when someone posts that blood has been arranged. Figure 2.1 shows this as-is process.

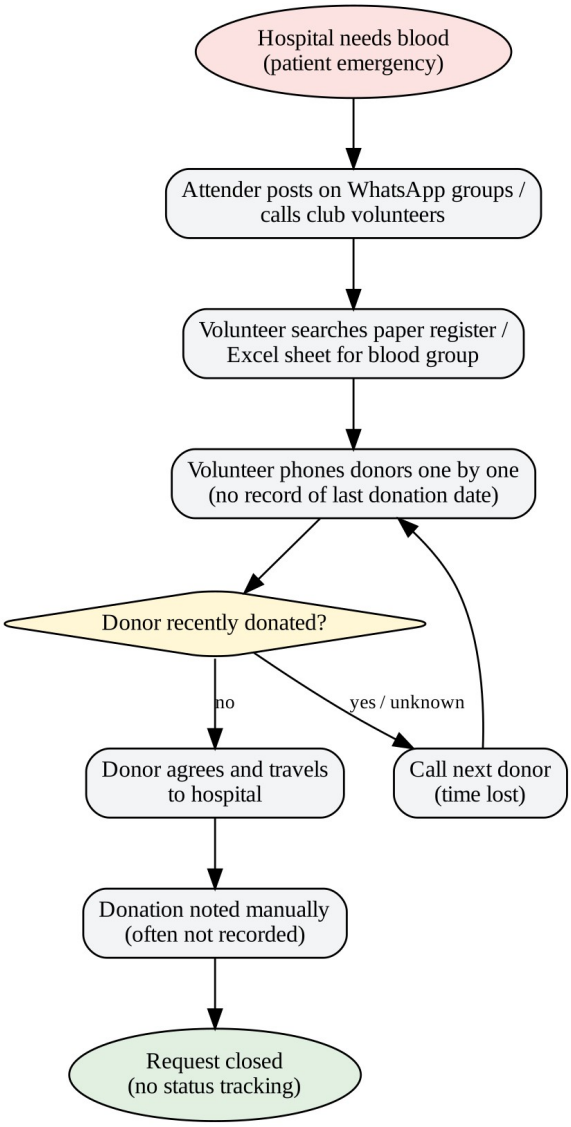


Figure 2.1: As-Is Process Flow of the Existing Manual Donor-Matching Process

2.2 Limitations of the Existing System

- Searching and calling donors is manual and slow; an urgent request can take several hours to fill.
- Eligibility (90-day deferral) and blood-group compatibility are not applied systematically, so eligible compatible donors are missed and ineligible donors are contacted.
- Data is duplicated across spreadsheets and chat groups, with inconsistent formats for names, phone numbers and blood groups.
- There is no status tracking of requests, no history of donations and no reporting on donor availability.
- Donors' personal data is shared widely in messaging groups with no access control.

2.3 Low-Code versus Traditional Development

The same application was compared as a traditional coded web app and as a Zoho Creator low-code app:

- **Development time:** about 6–8 weeks to code a database, backend, front end, login and email service, against about 1–2 weeks on Zoho Creator.
- **Hosting:** a coded app needs a server, database, SSL and backups; Zoho Creator is hosted and backed up by Zoho.
- **Security and mobile:** login, permission profiles and a mobile app come built in instead of being coded separately.
- **Changes:** fields and rules are changed visually and released through environments, so a student volunteer can maintain it.
- **Limits:** the interface is limited to platform components, which is enough for this project.

2.4 Literature Survey

- **Rosam and Kalnichevski [1]:** explain low-code concepts and when low-code is better than custom code; this shaped the forms–reports–workflows structure.
- **Bock and Frank [6]:** describe low-code platforms, citizen developers and their limits; used for the no-code / low-code / pro-code discussion.

- **Sahay et al. [7]:** compare eight low-code platforms and confirm that Zoho Creator has the data, workflow and deployment features needed.
- **Zoho documentation [2], [3]:** the main technical reference for forms, lookups, workflows and permissions.
- **WHO fact sheet [8] and e-RaktKosh [9]:** show the shortage of blood and that existing national portals serve blood banks, not volunteer donor networks.

2.5 Emerging Trends in Low-Code Development

Several low-code trends shaped the design. Citizen development fits here: a student volunteer can maintain the app after hand-over. Automation of matching and notification removes manual phone calls. AI-assisted building with Zia is available but was not used, so that every rule could be written and checked by hand.

2.6 Proposed System

The proposed system is a Zoho Creator application that replaces the spreadsheet and phone-tree process with a single shared data store and automated workflows. Each limitation in Section 2.2 is addressed directly: matching and notification are automated (manual calling); eligibility and compatibility are computed by Deluge (missed or ineligible donors); one validated form per entity removes duplication (scattered data); request status, donation history, reports and a dashboard give visibility (no tracking); and permission profiles restrict who can see donors' personal data (no access control).

CHAPTER 3
REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

- **Administrator:** owns the app; manages users, forms, workflows and releases.
- **Blood Bank Coordinator:** registers donors, logs requests and donations, and tracks request status.
- **Donor:** registers through the form and gets an email when a matching request arrives.
- **Viewer:** faculty in-charge or hospital contact who can only view reports and the dashboard.

3.2 Functional Requirements

- FR-01: Register a donor with name, email, phone, address, blood group, age and last donation date.
- FR-02: Reject a donor aged outside 18–65 or with a future donation date.
- FR-03: Mark a donor eligible only if they never donated or donated 90 or more days ago.
- FR-04: Log a blood request with patient, blood group, units, date, urgency and hospital.
- FR-05: Email every eligible donor whose blood group can donate to the patient.
- FR-06: Record each donation against a request and a donor.
- FR-07: Show donors by blood group and requests by urgency and status in reports.
- FR-08: Provide a dashboard with form shortcuts, a donor count and a blood-group chart.
- FR-09: Allow only the administrator and coordinators to edit records.

3.3 Non-Functional Requirements

- **Performance:** reports with up to 1,000 donors load within 3 seconds.
- **Security:** users see only what their permission profile allows.
- **Usability:** works on web and mobile with no training beyond Appendix D.
- **Auditability:** every record keeps who added or changed it and when.

3.4 Hardware and Software Requirements

- **Platform:** Zoho Creator (cloud) with Deluge scripting and Zoho Mail.
- **Client:** any PC with Google Chrome and internet; Zoho Creator mobile app on Android / iOS.

3.5 Use Case Model

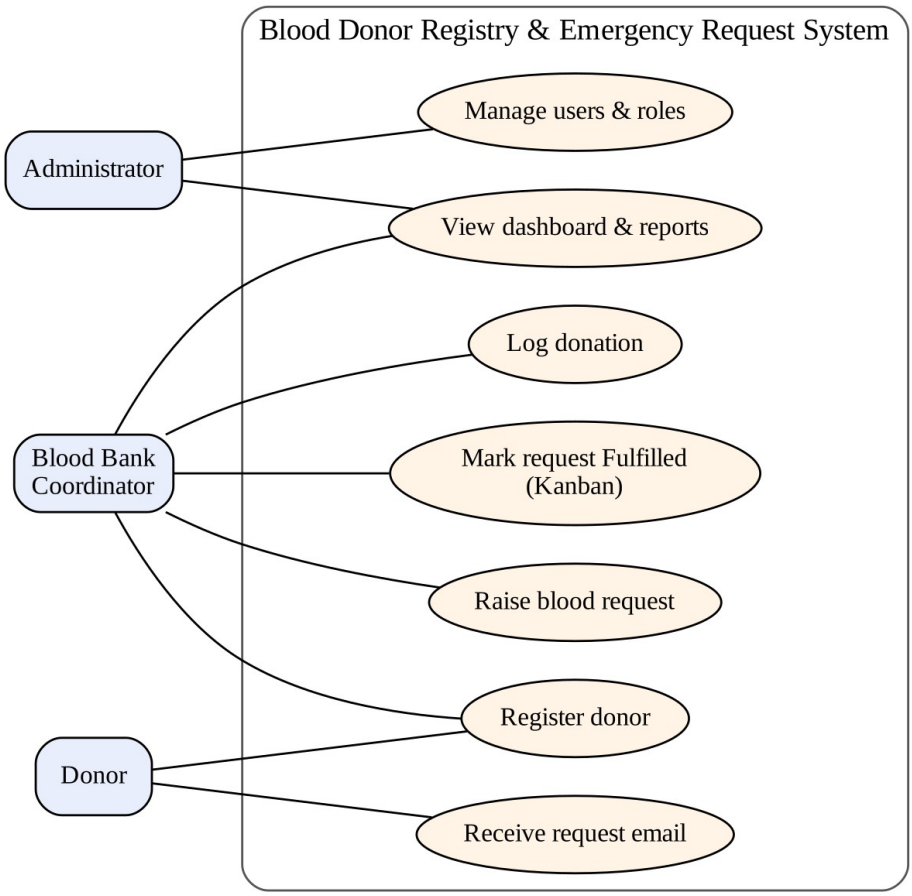


Figure 3.1: Use Case Diagram of the Blood Donor Registry

Two main use cases drive the application:

- **UC-01 Raise blood request:** the coordinator fills the Blood Request form; DS-02 finds eligible donors whose blood group matches and emails them, and the administrator gets a summary.
- **UC-02 Log donation:** the coordinator records the donation in the Donation Log, drags the request to Fulfilled on the Kanban board and updates the donor's last donation date.

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

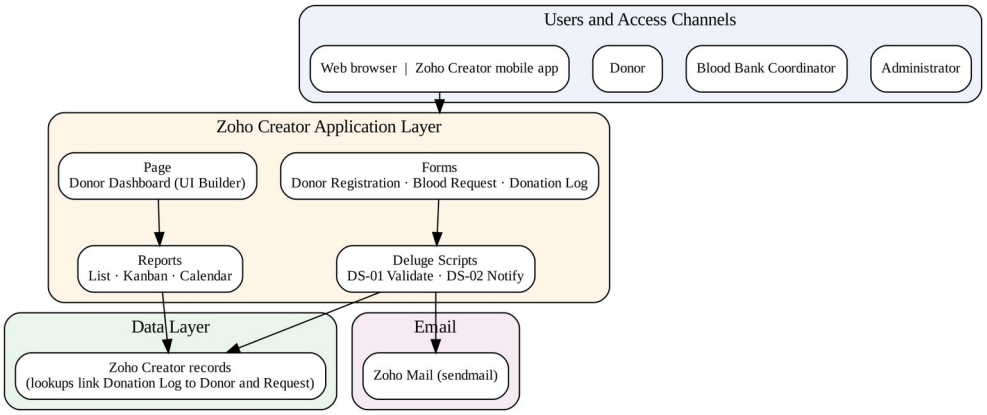


Figure 4.1: System Architecture of the Blood Donor Registry

The architecture has four layers. Users (administrator, coordinators, donors and viewers) use a browser or the Zoho Creator mobile app. The application layer holds the three forms, the reports, the Donor Dashboard and two Deluge scripts. The data layer is Zoho Creator's built-in database, where the Donation Log links to a donor and a request through lookup fields. Zoho Mail sends the notification emails.

4.2 Data Model

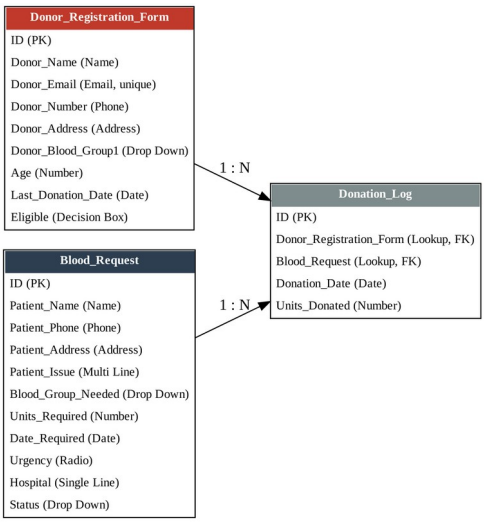


Figure 4.2: Entity–Relationship Diagram of the Application

The three forms are linked as shown in Figure 4.2. A donor can donate many times and a request can be met by several donors, so the Donation Log sits between them with one lookup to each.

4.3 Form Design

- **Donor Registration:** Name, Email (no duplicates), Phone, Address, Blood Group (8-value drop-down), Age, Last date of donation, Eligible (tick box set by DS-01).
- **Blood Request:** Patient Name, Phone, Address, Issue, Blood Group Needed, Units, Date Required, Urgency (Normal / Urgent), Hospital, Status (Open / Fulfilled).
- **Donation Log:** lookup to Blood Request, lookup to Donor, Donation Date, Units Donated.

4.4 Process and Workflow Design

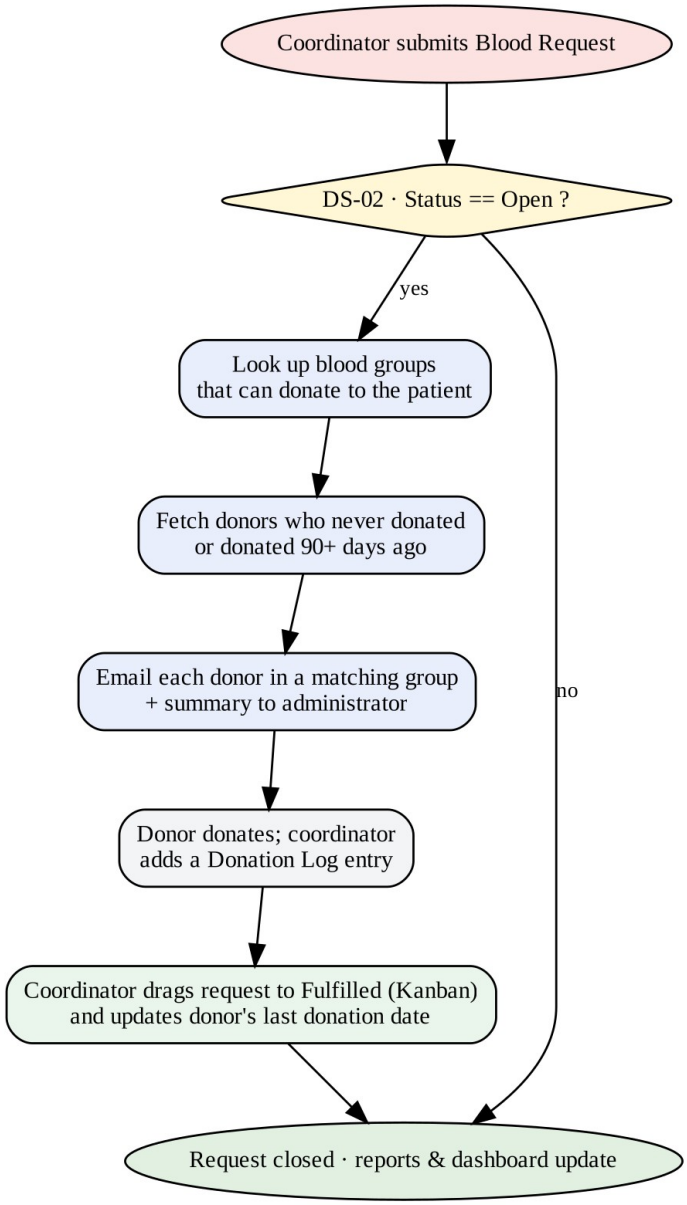


Figure 4.3: Process Flow for Blood Request Matching and Donation

4.5 User Interface and Navigation Design

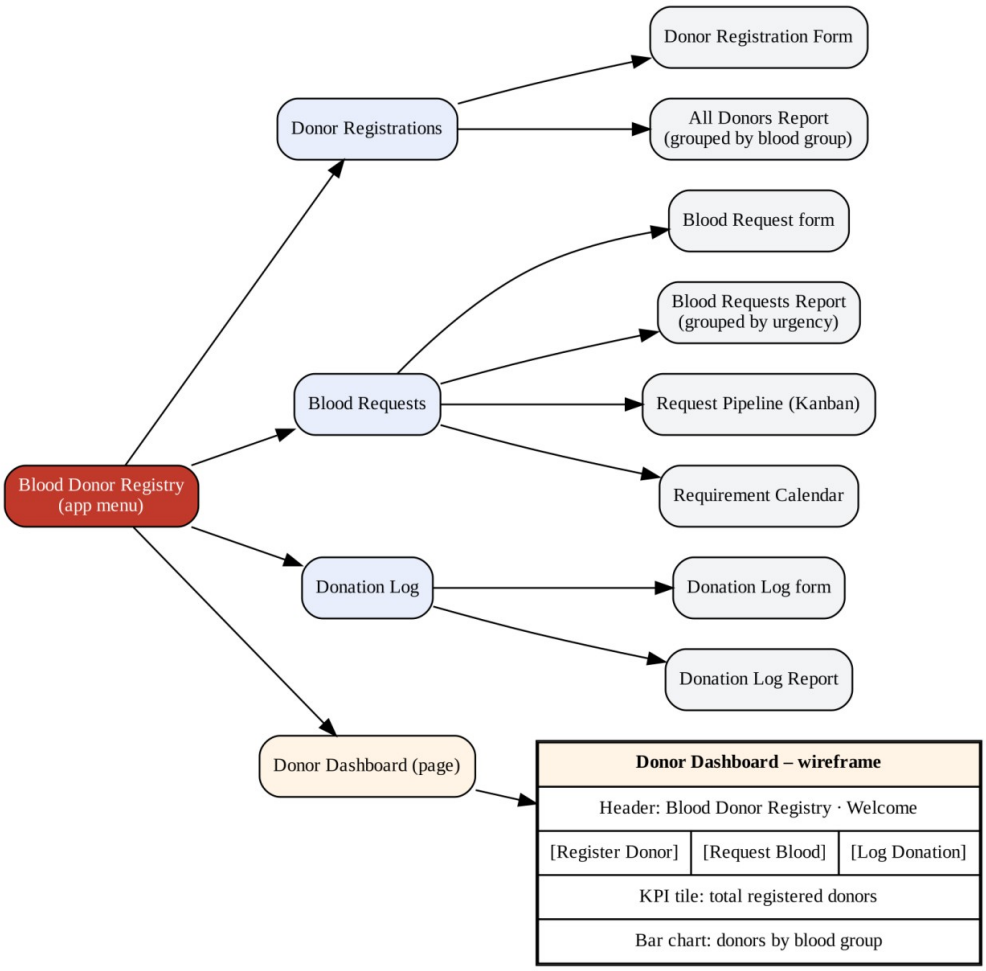


Figure 4.4: Navigation Structure and Wireframe of the Donor Dashboard

4.6 Access Control Design

- **Administrator:** full access to all forms, reports, settings and deployment.
- **Coordinator (Write profile):** add, edit and view all three forms and reports.
- **Viewer (Read profile):** view reports and dashboard only.
- **Donor:** can only fill the Donor Registration form.

CHAPTER 5
IMPLEMENTATION

5.1 Application Setup

The application was created from scratch in the Zoho Creator workspace adishwarseelan.sk.2024.cs215 (Create Application → From Scratch) and named “Blood Donor Registry & Emergency Request System”. It is developed in the Development environment, with Stage and Production available for release. The application menu is organised into three sections that mirror the three entities — Donor Registrations, Blood Requests and Donation Log — each holding its form and reports, followed by the Donor Dashboard page. The default theme was retained; the menu was arranged so that coordinators reach the Blood Requests section, which they use most, in one click.

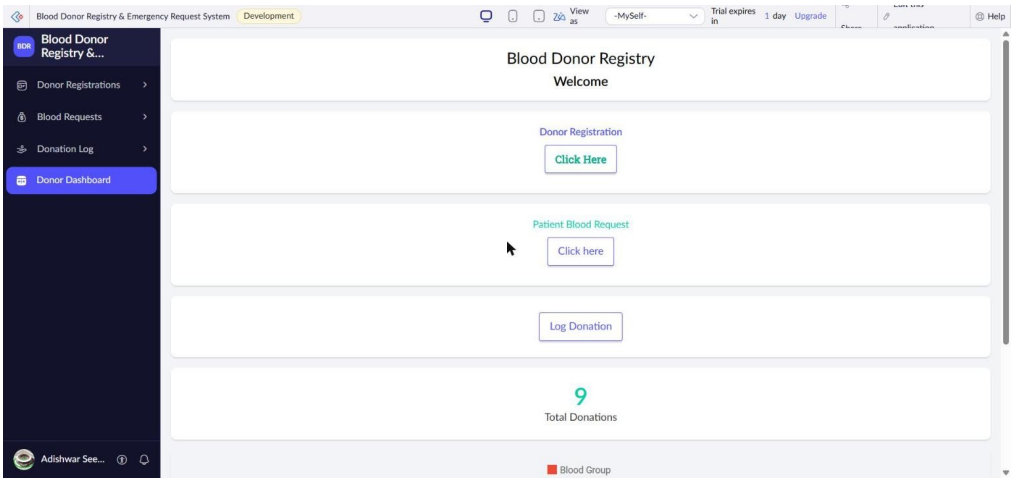


Figure 5.1: Application Home (Donor Dashboard) and Navigation Sections

5.2 Forms and Data Capture

Three forms were built with the form builder from blank forms. The Donor Registration form uses Name, Email (no duplicate values), Phone, Address (with geo-capture), Drop Down, Number, Date, Decision Box and Single Line fields — nine field types in one form. Blood group is a Drop Down with eight fixed values so that matching cannot fail on spelling. The Blood Request form adds Multi Line and Radio fields and a Status drop-down with Open as the default value. The Donation Log form connects the entities through two single-select lookup fields to the Blood Request and Donor Registration forms. Field-level validations are configured as mandatory flags and unique constraints, and the business validations (age 18–65, no future donation

date) run in the On Validate workflow DS-01. Each form has a custom success message, for example “Donor Registration Added Successfully”.

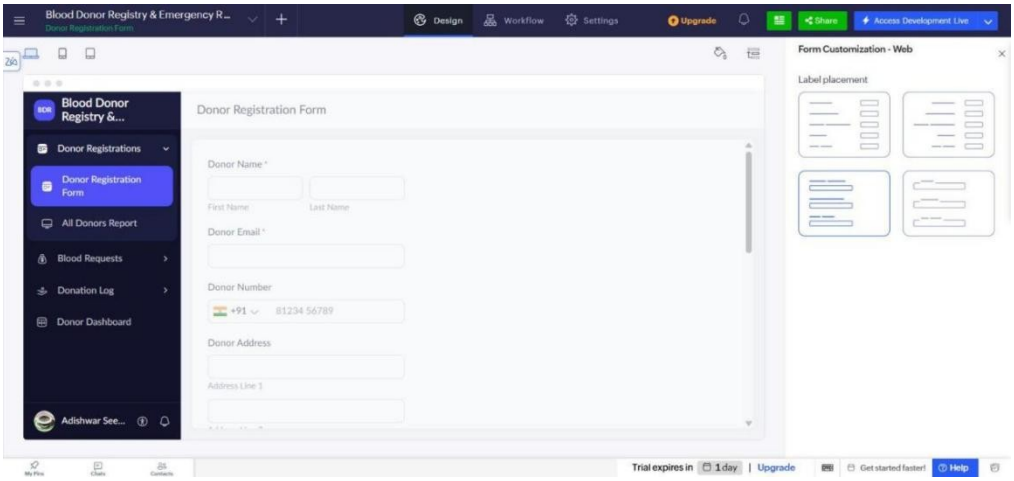


Figure 5.2: Donor Registration Form in the Form Builder

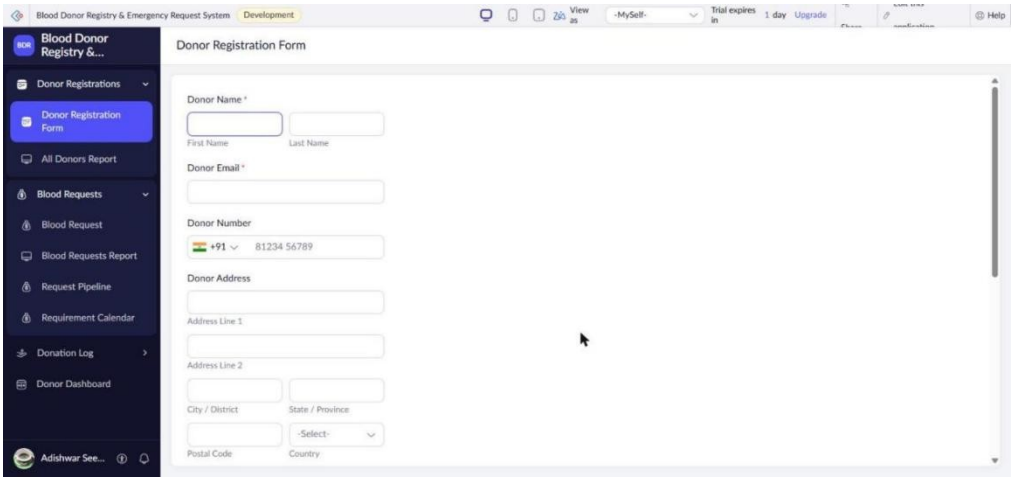


Figure 5.3: Donor Registration Form in Live Mode

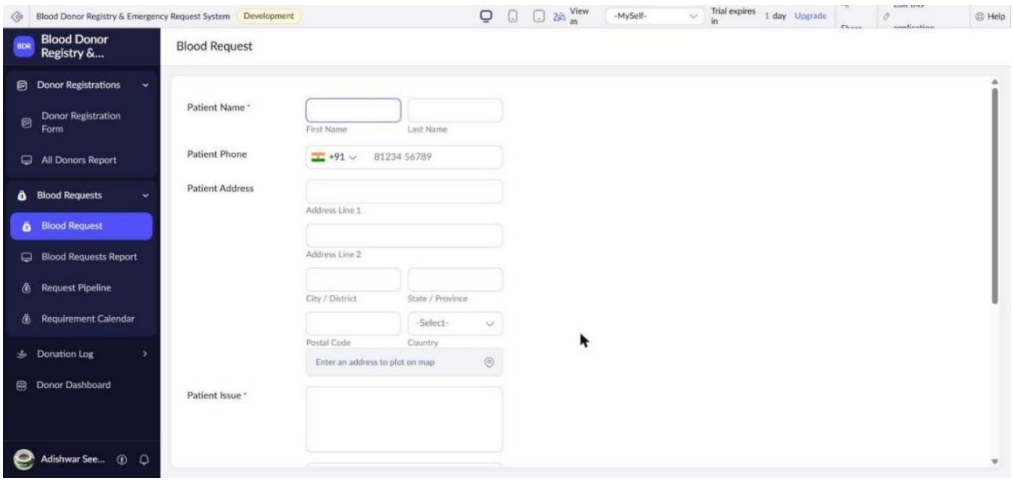


Figure 5.4: Blood Request Form in Live Mode

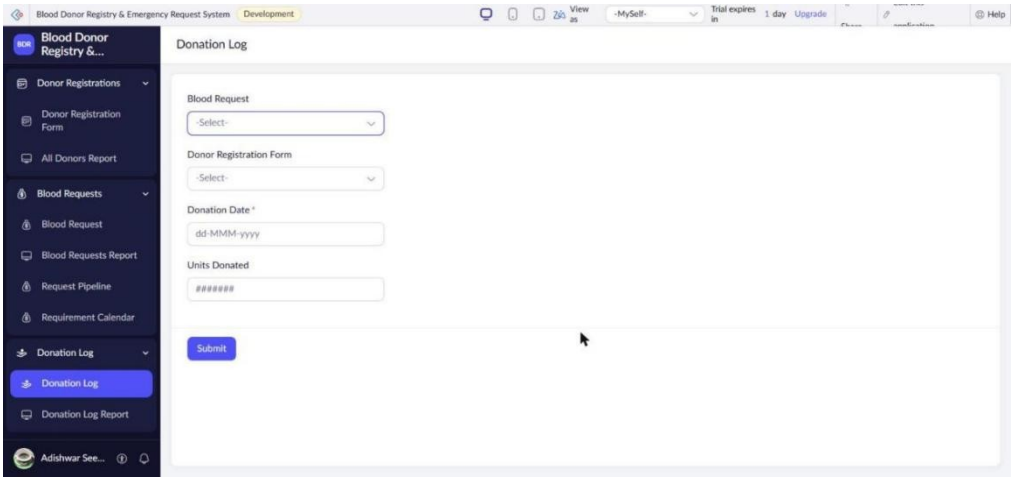


Figure 5.5: Donation Log Form with Lookup Fields to Blood Request and Donor

5.3 Reports

- **All Donors Report (list):** grouped by blood group with counts and quick filters.
- **Blood Requests Report (list):** grouped by urgency, urgent first, sorted by date required.
- **Request Pipeline (Kanban):** one card per request in Open or Fulfilled columns; drag to change status.
- **Requirement Calendar:** shows each request on the date blood is needed.

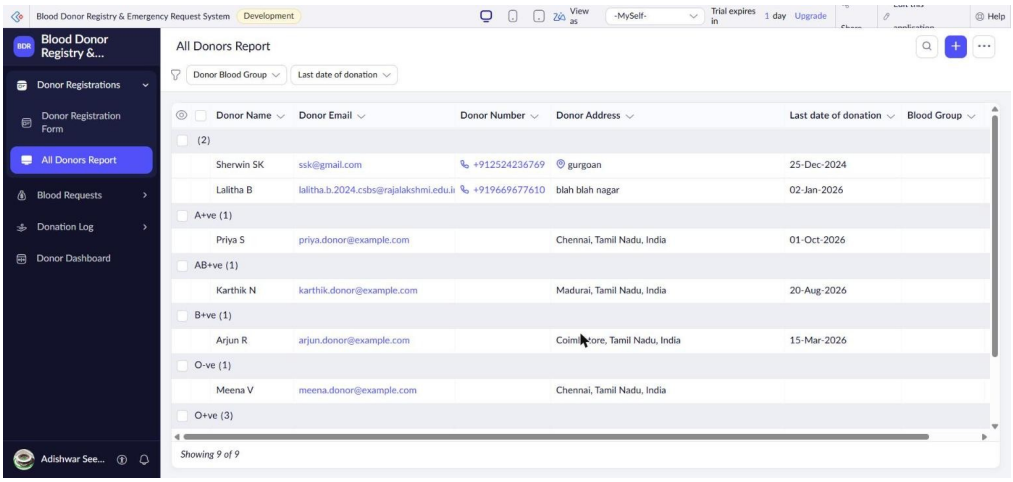


Figure 5.6: All Donors Report Grouped by Blood Group with Quick Filters

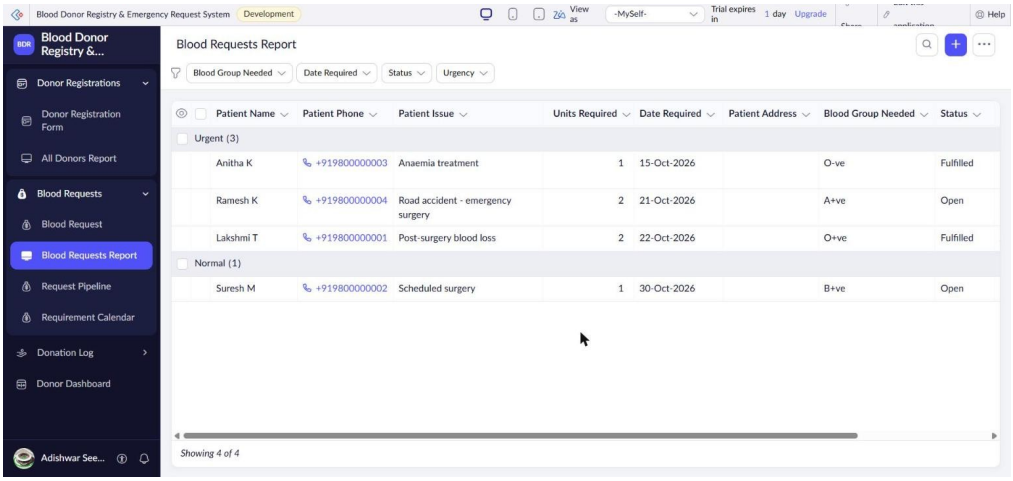


Figure 5.7: Blood Requests Report Grouped by Urgency and Sorted by Date Required

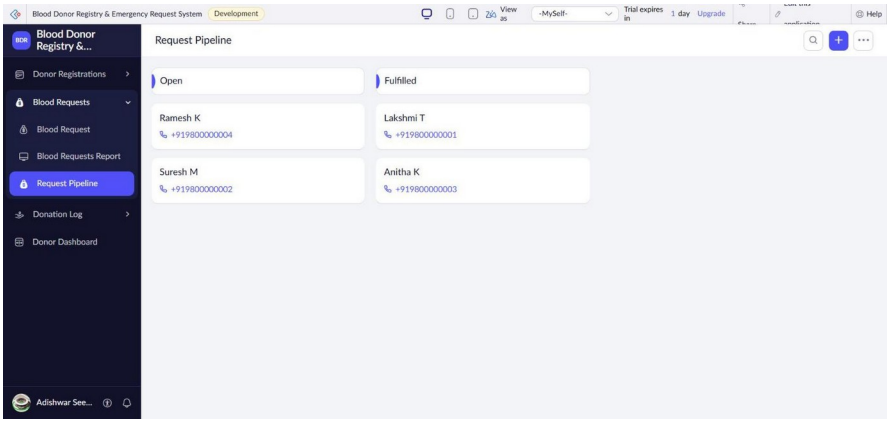


Figure 5.8: Request Pipeline – Kanban Report by Status

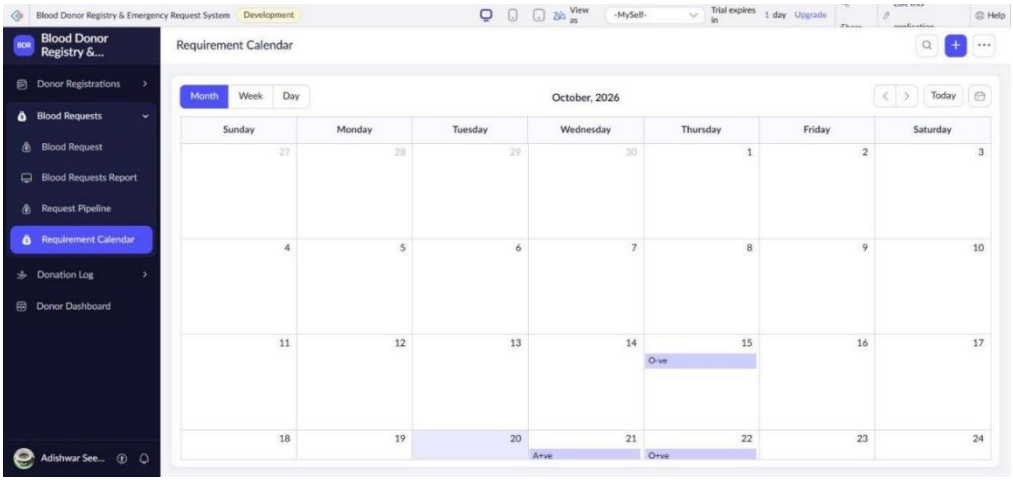


Figure 5.9: Requirement Calendar Report by Date Required

5.4 Pages and Dashboards

The Donor Dashboard page was built with the UI Builder for coordinators and viewers. It contains a header panel, three button panels that open the Donor Registration, Blood Request and Donation Log forms, a KPI panel showing the count of registered donor records, and a bar chart of the number of donors in each blood

group drawn from the Donor Registration form. During testing the chart was found to plot an unused field and show only “Unknown”; its category and aggregate were corrected to Donor Blood Group (Defect BUG-01).

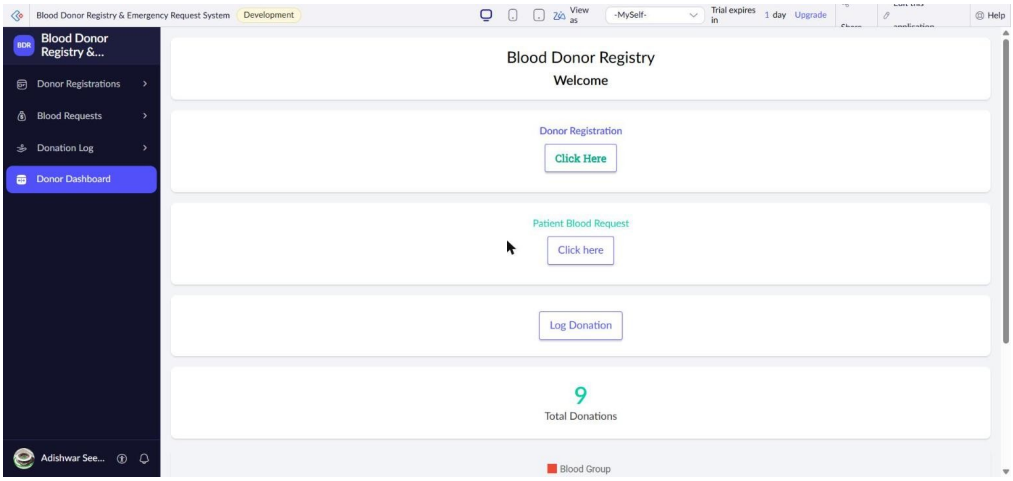


Figure 5.10: Donor Dashboard Page – Navigation Buttons and KPI Panel

5.5 Deluge Scripting

Only two Deluge scripts are used; everything else is configured without code.

- **DS-01 (Donor Registration – On Validate):** blocks donors outside age 18–65 or with a future donation date, and ticks Eligible when the last donation is 90 or more days ago.
- **DS-02 (Blood Request – On Success):** looks up which blood groups can donate to the patient, emails each eligible donor in those groups and sends the administrator a summary.

Code 5.1: DS-01 – Donor validation and eligibility (On Validate)

```
// DS-01 | Donor Registration Form | Created or Edited -> Validations
on form submission
// Purpose: validate donor age and donation date, then compute
eligibility (90-day rule)
if(input.Age < 18 || input.Age > 65)
{
    alert "Donor age must be between 18 and 65 years.";
    cancel submit;
}
if(input.Last_Donation_Date != null && input.Last_Donation_Date >
zoho.currentdate)
{
    alert "Last donation date cannot be in the future.";
    cancel submit;
}
if(input.Last_Donation_Date == null)
{
    input.Eligible = true;
}
else
```

```
{
    input.Eligible =
input.Last_Donation_Date.daysBetween(zoho.currentdate) >= 90;
}
```

Code 5.2: DS-02 – Compatible donor matching and notification (On Success)

```
// DS-02 | Blood Request | Created -> Successful form submission
// Purpose: email every eligible donor whose blood group can donate to
the patient
if(input.Status == "Open")
{
    compat = Map();
    compat.put("O-ve",{"O-ve"});
    compat.put("O+ve",{"O-ve","O+ve"});
    compat.put("A-ve",{"O-ve","A-ve"});
    compat.put("A+ve",{"O-ve","O+ve","A-ve","A+ve"});
    compat.put("B-ve",{"O-ve","B-ve"});
    compat.put("B+ve",{"O-ve","O+ve","B-ve","B+ve"});
    compat.put("AB-ve",{"O-ve","A-ve","B-ve","AB-ve"});
    compat.put("AB+ve",{"O-ve","O+ve","A-ve","A+ve","B-ve","B+ve","AB-
ve","AB+ve"});
    groups = compat.get(input.Blood_Group_Needed);
    cutoff = zoho.currentdate.subDay(90);
    donors = Donor_Registration_Form[Last_Donation_Date == null ||
Last_Donation_Date <= cutoff];
    notified = 0;
    for each d in donors
    {
        if(groups.contains(d.Donor_Blood_Group1) && d.Donor_Email !=
null)
        {
            sendmail
            [
                from :zoho.adminuserid
                to :d.Donor_Email
                subject :input.Urgency + " blood request: " +
input.Blood_Group_Needed + " needed"
                message :"Dear " + d.Donor_Name.first_name +
",<br><br>A patient at " + input.Hospital + " needs " +
input.Units_Required + " unit(s) of " + input.Blood_Group_Needed + "
blood by " + input.Date_Required + ".<br>Your blood group (" +
d.Donor_Blood_Group1 + ") is compatible. Please reply if you can
donate.<br><br>Blood Donor Registry"
            ]
            notified = notified + 1;
        }
    }
    info "Donors notified: " + notified;
    sendmail
    [
        from :zoho.adminuserid
        to :zoho.adminuserid
        subject :"Request logged: " + input.Blood_Group_Needed + " ("
+ input.Urgency + ")"
        message :"A new blood request for " + input.Patient_Name + "
was logged. Eligible compatible donors notified: " + notified
    ]
}
```

DS-02 checks the 90-day rule directly from the last donation date, so a donor becomes available again automatically without a scheduled job. For an “A+ve” request it

emails eligible O-ve, O+ve, A-ve and A+ve donors. The complete code is in Appendix C.

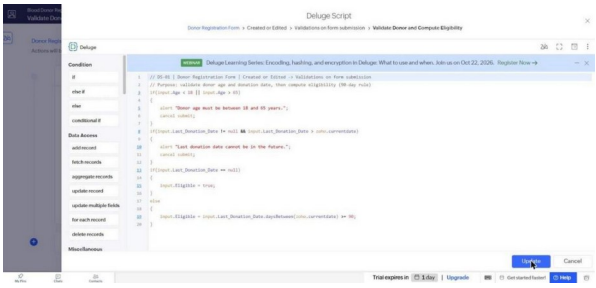


Figure 5.11: DS-01 in the Deluge Editor (Donor Registration – On Validate)

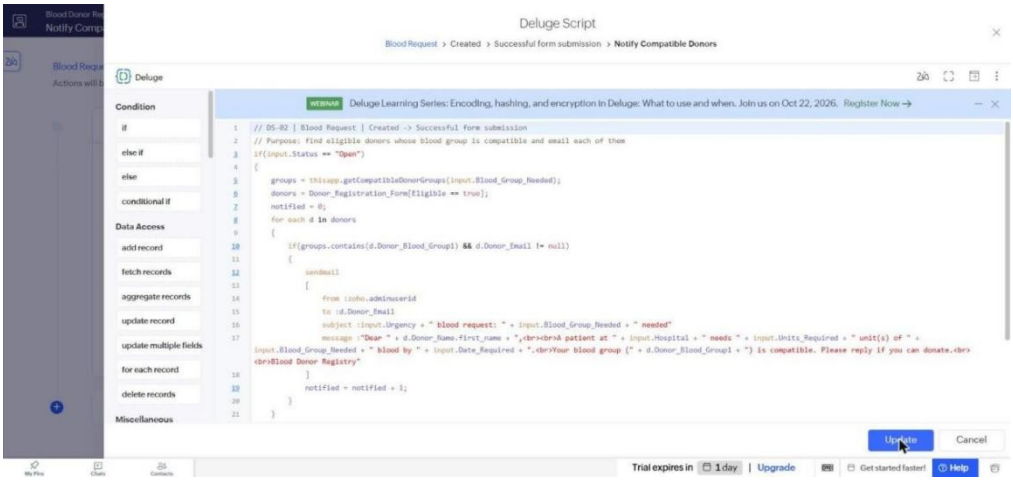


Figure 5.12: DS-02 in the Deluge Editor (Blood Request – On Success)

5.6 Workflows and Automation

Two form workflows run the scripts. The rest of the process is handled without code:

- **Validate Donor:** Donor Registration, on create or edit, runs DS-01.
- **Notify Compatible Donors:** Blood Request, on create, runs DS-02.
- **Closing a request:** done by dragging the card to Fulfilled on the Kanban board — no script needed.

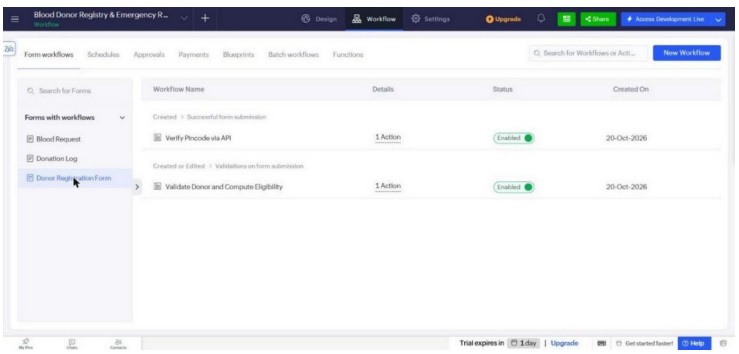


Figure 5.13: Form Workflows Configured on the Donor Registration Form

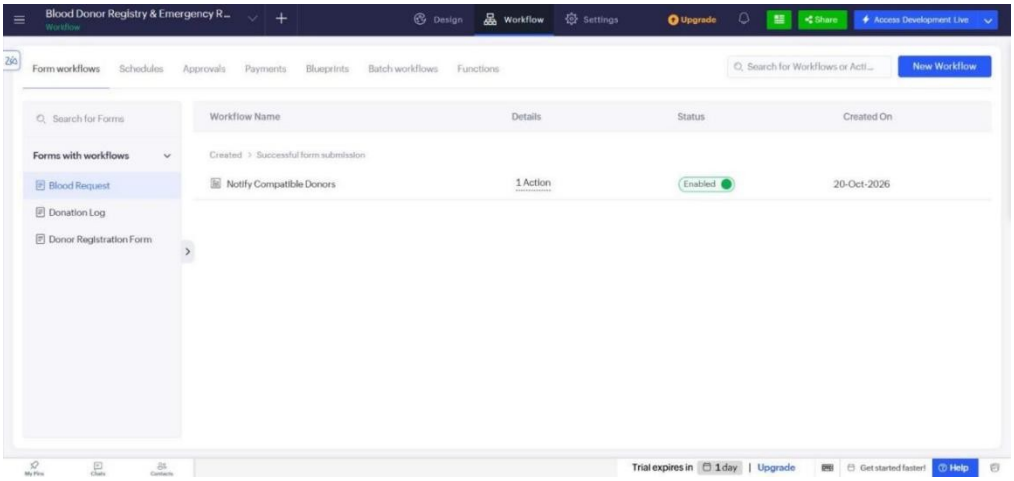


Figure 5.14: Form Workflow Configured on the Blood Request Form

5.7 Integrations

Zoho Mail is used through the built-in sendmail task in DS-02, so no external connection or API key is needed. During testing, sample records were added and checked through the Creator REST API.

5.8 User Management and Role-Based Access

Users are managed under Settings → User Permissions. Coordinators get the Write profile (add, edit, view) and viewers get the Read profile (view only), as shown in Figure 5.15. The administrator owns the app and has full access.

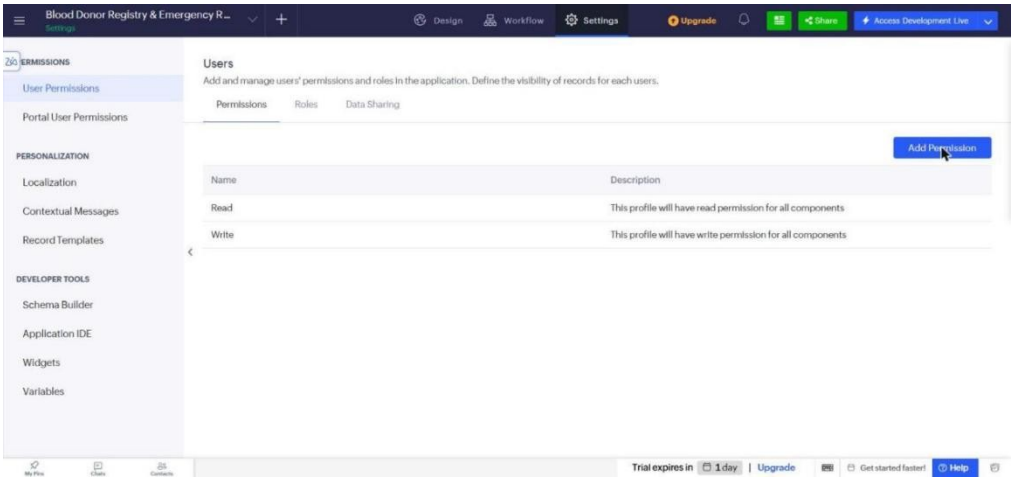


Figure 5.15: Permission Profiles under Settings → User Permissions

5.9 AI and Analytics Features

Analytics are provided by the bar chart on the Donor Dashboard, which counts donors per blood group directly from the Donor Registration form, and by grouped reports that show record counts per blood group and per urgency. Zia AI fields (prediction, OCR, sentiment, keyword extraction) were reviewed but not used, because the

application has no historical data to train a prediction and no unstructured text or images to analyse; no part of the application logic was AI-generated. A pivot table/chart was tried and requires a Zoho Analytics workspace, which was not configured on this account.

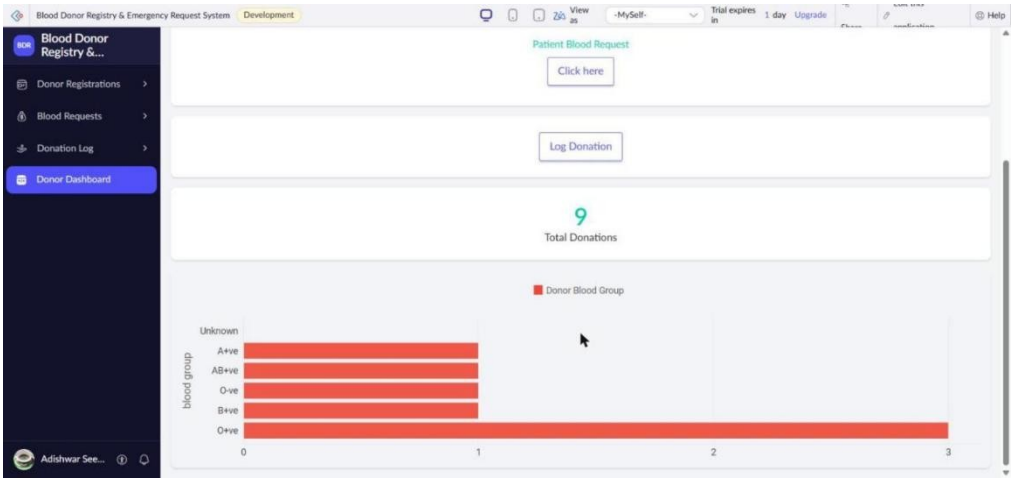


Figure 5.16: Analytics – Bar Chart of Donors by Blood Group on the Donor Dashboard

5.10 Pro-Code Extension

The application stays fully within low-code: two short Deluge scripts and no external code. A Zoho Catalyst (pro-code) function would only be needed for something like ranking donors by distance in a very large registry.

5.11 Mobile and Portal Access

The application is available in the Zoho Creator mobile app without additional configuration; forms and reports adapt automatically to the phone layout. No separate mobile layout or customer portal was configured, because coordinators use the web interface and donors register through the shared form. A patient-facing portal is listed as a future enhancement.

CHAPTER 6
TESTING AND DEBUGGING

6.1 Testing Strategy

Testing was done in the Development environment. Each script was tested by submitting real records through the live forms, including invalid data such as an over-age donor, and the reports and dashboard were checked after each step.

6.2 Test Cases

Table 6.1: Test Cases

TC ID	Test Description	Expected Result	Status
TC-01	Register a donor with all mandatory fields	Record saved with success message	Pass
TC-02	Register a donor aged 70	Alert shown; record not saved	Pass
TC-03	Submit donor form with email missing	Required-field error shown	Pass
TC-04	Donor who donated 25 days ago	Eligible = false	Pass
TC-05	New donor with no donation date	Eligible = true	Pass
TC-06	Log an urgent A+ve request	Saved as Open; DS-02 runs	Pass
TC-07	Check donor emails for that request	Only compatible eligible donors and admin get email	Pass
TC-08	Log a donation and drag the request to Fulfilled	Request shows in Fulfilled column	Pass
TC-09	Open donor and request reports	Grouped by blood group / urgency with counts	Pass
TC-10	Open the Donor Dashboard	Bar for each blood group	Pass
TC-11	Read-only user tries to edit a request	Edit not allowed	Pass

6.3 Debugging Deluge Scripts

The Deluge editor checks the code on every save and shows the line of any error, so each script was saved only after it compiled cleanly. An info statement in DS-02 prints the number of donors notified in the execution log, which confirms each run.

6.4 Defect Log

- **BUG-01:** the dashboard chart showed only “Unknown”; fixed by pointing the chart to the Donor Blood Group field.
- **BUG-02:** blood group typed as free text could not be matched; fixed by changing it to a drop-down with eight values.

- **BUG-03:** patient phone set to “no duplicates” blocked repeat requests; fixed by removing that setting.

6.5 User Acceptance Testing

Sherwin S K (coordinator) found registering donors and logging requests quick and asked for urgent requests to stand out, so the request report was grouped with Urgent first. Lalitha B (viewer) found the dashboard and Kanban easy to read and confirmed that editing was blocked for her.

CHAPTER 7
DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

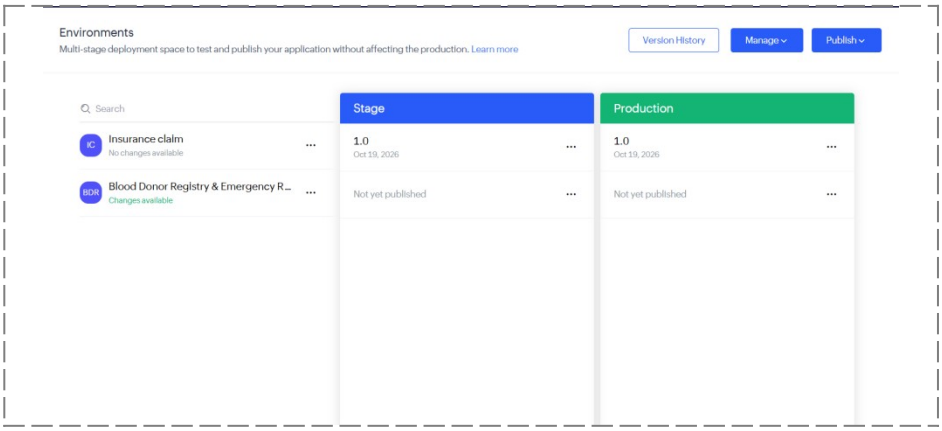


Figure 7.1: Environments – Publish to Stage and Production

All work was done in the Development environment. The release plan is: publish to Stage, re-test with invited users, then publish to Production for coordinators. Version history allows a rollback if a problem is found.

Deployment checklist:

- ✓ All test cases executed
- ✓ Test data kept in Development only
- ✓ Users and permissions configured
- ✓ Workflows enabled
- ✓ Application published and URL verified
- ✓ Release notes recorded

7.2 Performance Optimisation

DS-02 fetches the eligible donors with a single criteria fetch instead of one fetch per blood group, and reports use grouping and quick filters instead of a separate report for each blood group.

7.3 Security Features

- **Permission profiles:** only the administrator and coordinators can change data.
- **Validation:** mandatory fields, unique email and DS-01 stop bad donor records.
- **No secrets in scripts:** email uses the built-in sendmail task.

- **Separate environments:** untested changes never reach live users.

7.4 Governance and Administration

Every record stores who added or changed it and when, which gives a basic audit trail. Scripts are named DS-01 and DS-02 with a header comment, and any report can be exported to CSV or Excel as a backup.

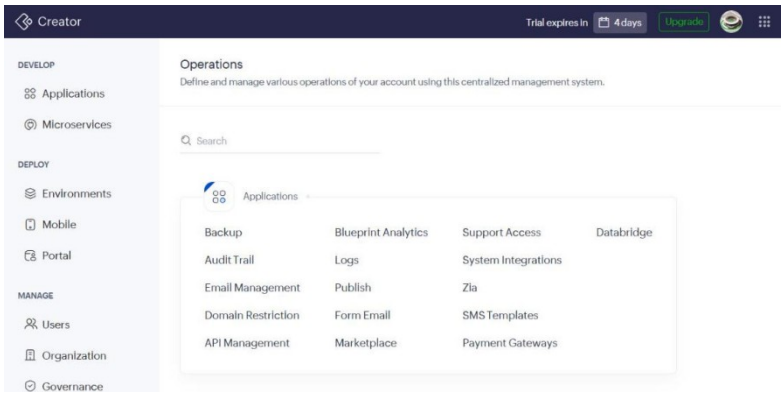


Figure 7.2: Audit Trail / Administration Settings

CHAPTER 8
RESULTS AND DISCUSSION

8.1 Application Output

Step 1: donors are registered and DS-01 sets their eligibility; the All Donors Report groups them by blood group. Step 2: an urgent A+ve request for Ramesh K is logged and DS-02 emails eligible O-ve, O+ve, A-ve and A+ve donors. Step 3: a donation is logged for Lakshmi T's request and the card is dragged to Fulfilled on the Kanban board, while Ramesh K's request stays Open.

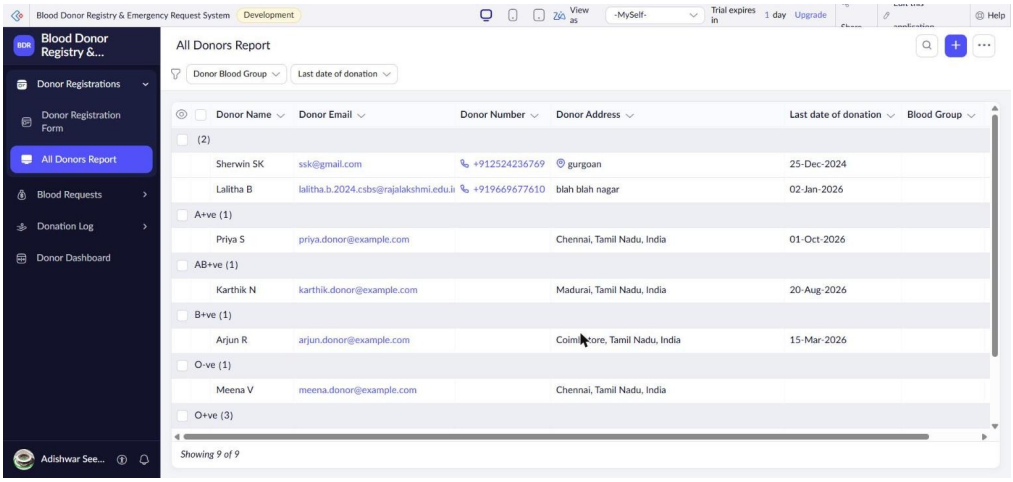


Figure 8.1: Scenario Step 1 – Donors Registered and Grouped by Blood Group

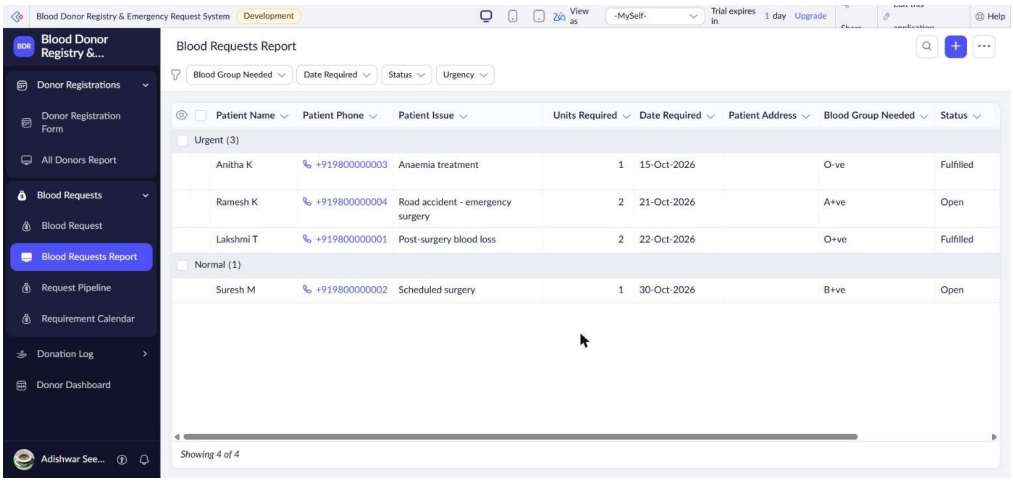


Figure 8.2: Scenario Step 2 – Urgent A+ve Request Logged for Ramesh K

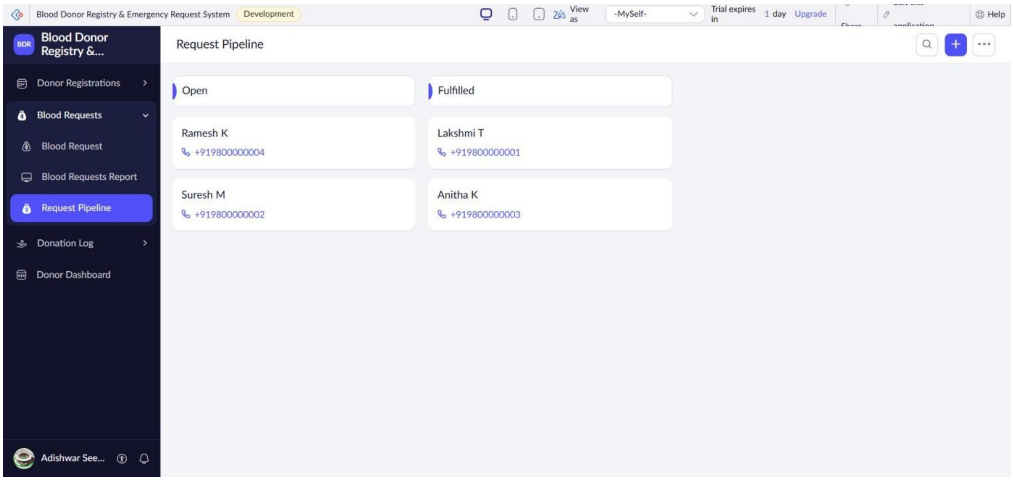


Figure 8.3: Scenario Step 3 – Donation Logged and Request Moved to Fulfilled

8.2 Achievement of Objectives

All five objectives in Section 1.3 were met: the three linked forms, the two scripts for eligibility and notification, the reports and dashboard, email through Zoho Mail, and the Development → Stage → Production release plan with permission profiles.

8.3 Comparison with the Existing System

- **Finding donors:** hours of phone calls before; automatic email on saving the request now.
- **Eligibility and blood group:** asked over the phone before; checked by the scripts for every donor now.
- **Reports:** none before; grouped reports, Kanban, Calendar and a chart now.
- **Data:** spreadsheets and chat groups before; one shared database with access control now.

8.4 Limitations of the Low-Code Approach

Report grouping and filters can only be set by hand in the builder; pivot reports need a separate Zoho Analytics workspace; sendmail has daily email limits on lower plans; and the app is tied to Zoho Creator (vendor lock-in).

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

No-code built most of the app: forms, lookups, reports, the Kanban, the dashboard, permissions and environments. Low-code Deluge was needed only for two rules — donor validation and matching donors by blood group. Pro-code would only be needed for things like distance-based ranking or SMS through an external gateway.

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

This project replaced phone-based donor matching with a Zoho Creator app that registers donors, captures blood requests, emails eligible compatible donors automatically and tracks each request to Fulfilled. Two Deluge scripts handle the rules; the rest is no-code configuration.

All test cases passed, three defects were found and fixed, and the app is ready for staged release.

9.2 Future Enhancements

- Blueprint for the request lifecycle (Open → Donors Contacted → Fulfilled / Cancelled) with mandatory notes at each transition.
- SMS / WhatsApp notification through a messaging gateway connection for donors who do not read email.
- Distance-based donor ranking using captured map coordinates, implemented as a Zoho Catalyst function.
- A customer portal for patients' attenders to raise and track requests themselves.
- Zia prediction of donor response probability once enough donation history is collected, and integration with Zoho Analytics for monthly donation statistics.

9.3 Learning Outcomes and Career Relevance

The project built practical skills in modelling related data, configuring forms, reports and pages, writing and debugging Deluge, setting permissions and planning a staged release — skills used by low-code developers and Zoho Creator consultants.

REFERENCES

[1] W. Rosam and O. Kalnichevski, Low-Code Development: A Practical Guide. New Delhi, India: BPB Publications.

[2] Zoho Corporation, “Zoho Creator Help Documentation.” [Online]. Available: https://www.zoho.com/creator/help/ (Accessed: 20 October 2026).

[3] Zoho Corporation, “Deluge Script Reference Guide.” [Online]. Available: https://www.zoho.com/deluge/ (Accessed: 20 October 2026).

[4] Zoho Corporation, “Zoho Creator Learning Centre.” [Online]. Available: https://www.zoho.com/creator/resources/ (Accessed: 20 October 2026).

[5] Gartner, Inc., “Magic Quadrant for Enterprise Low-Code Application Platforms,” Gartner Research, 2023.

[6] A. C. Bock and U. Frank, “Low-Code Platform,” Business & Information Systems Engineering, vol. 63, no. 6, pp. 733–740, 2021.

[7] A. Sahay, A. Indamutsa, D. Di Ruscio and A. Pierantonio, “Supporting the understanding and comparison of low-code development platforms,” in Proc. 46th Euromicro Conf. Software Engineering and Advanced Applications (SEAA), 2020, pp. 171–178.

[8] World Health Organization, “Blood safety and availability,” Fact sheet. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/blood-safety-and-availability

[9] Ministry of Health and Family Welfare, Government of India, “e-RaktKosh.” [Online]. Available: https://eraktkosh.mohfw.gov.in/

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

Table A.1: Feature Coverage Checklist

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
I	Application created & sections/menu organised	✓	App menu with 3 sections + Donor Dashboard	5.1	Fig 5.1
II	Forms with ≥ 8 distinct field types	✓	Donor Registration: Name, Email, Phone, Address, Drop Down, Number, Date, Decision Box, Single Line; plus Multi Line, Radio, Lookup	5.2	Fig 5.2–5.5
II	Lookup field (relationship between forms)	✓	Donation Log → Blood Request and Donor	4.2, 5.2	Fig 4.2, 5.5
II	Subform (parent–child data)	✗	Not required (Section 4.2)	4.2, 5.2	—
II	Field validations / layout rules	✓	Mandatory, unique email, DS-01 On Validate	5.2	Code 5.1
II	List report with filter, grouping, custom action	✓	All Donors / Blood Requests reports (grouping, quick filters)	5.3	Fig 5.6, 5.7
II	Kanban / Calendar / Timeline / Map report	✓	Request Pipeline (Kanban), Requirement Calendar	5.3	Fig 5.8, 5.9
II	Pivot table / chart report	✓	Bar chart on Donor Dashboard; pivot needs Zoho Analytics	5.3	Fig 5.16
II	Page / dashboard built with UI Builder	✓	Donor Dashboard	5.4	Fig 5.10
II	Deluge – form events (validate / success / field actions)	✓	DS-01, DS-02	5.5	Code 5.1, 5.2, Fig 5.11–5.12

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
II	Deluge – functions, fetch, loops, collections	✓	Map/List, criteria fetch, for each in DS-02	5.5	Code 5.2
II	Email / in-app notifications	✓	sendmail in DS-02	5.5, 5.6	Appendix C
II	Form workflow	✓	2 form workflows	5.6	Fig 5.13, 5.14
II	Scheduled workflow	✗	Not needed – DS-02 checks the 90-day rule	5.6	—
II	Approval process	✗	Not used	5.6	—
II	Blueprint	✗	Planned (9.2)	5.6	—
II	Batch / report workflow	✗	Not used	5.6	—
II	Testing & debugging evidence	✓	Test cases and defect list	6	Table 6.1
III	Integration with a Zoho service	✓	Zoho Mail via sendmail	5.7	Section 5.7
III	Connection / REST API call (invokeurl)	✗	Not used	5.7	—
III	Webhook or Creator REST API	✓	Creator REST API v2 used for test data	5.7	Section 5.7
III	Users, roles and permission profiles	✓	Read / Write profiles	5.8	Fig 5.15
III	Record-level access criteria / customer portal	✗	Planned for production	5.8	—
III	Zia / AI feature	✗	Reviewed, not applicable	5.9	—
III	Analytics (charts / Zoho Analytics)	✓	Dashboard chart	5.9	Fig 5.16
IV	Performance optimisation applied & measured	✓	Single criteria fetch in DS-02	7.2	Section 7.2
IV	Deployment across environments	✓	Development → Stage → Production	7.1	Fig 7.1
IV	Security controls (encryption, OAuth, permissions)	✓	Profiles, validation, no secrets	7.3	Section 7.3

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
IV	Governance (audit trail, administration)	✓	Added/Modified user and time, naming, export	7.4	Fig 7.2
V	Pro-code extension (Catalyst / widgets / custom functions)	✗	Not required; Catalyst justified	5.10	—
V	Low-code limitations & spectrum analysis	✓	Limitations and spectrum analysis	8.4, 8.5	—
—	Mobile app / portal access	✓	Available in Zoho Creator mobile app	5.11	—

Total features implemented: 23 / 32

APPENDIX B – COURSE OUTCOME MAPPING

Table B.1: Mapping of Course Outcomes to Report Chapters

CO	Course Outcome Statement	Evidenced In	Self-Assessed Level
CO1	Explain the principles, architecture, and business value of low-code application development using Zoho Creator.	Ch. 1, 2	3
CO2	Develop business applications using Zoho Creator forms, reports, workflows, and Deluge scripting.	Ch. 4, 5 (5.1–5.6), 6	3
CO3	Integrate Zoho Creator applications with Zoho services and external systems using APIs, connections, and role-based security.	Ch. 4.6, 5.7–5.9	2
CO4	Deploy, secure, and manage Zoho Creator applications across development and production environments following governance best practices.	Ch. 7	2
CO5	Evaluate low-code development strategies, AI-assisted application development, and the transition from citizen developer to professional developer.	Ch. 5.9, 5.10, 8.4, 8.5, 9.3	3

APPENDIX C – COMPLETE DELUGE SOURCE CODE

C.1 Donor Registration Form – DS-01 (On Validate)

```
// DS-01 | Donor Registration Form | Created or Edited -> Validations
on form submission
// Purpose: validate donor age and donation date, then compute
eligibility (90-day rule)
if(input.Age < 18 || input.Age > 65)
{
    alert "Donor age must be between 18 and 65 years.";
    cancel submit;
}
if(input.Last_Donation_Date != null && input.Last_Donation_Date >
zoho.currentdate)
{
    alert "Last donation date cannot be in the future.";
    cancel submit;
}
if(input.Last_Donation_Date == null)
{
    input.Eligible = true;
}
else
{
    input.Eligible =
input.Last_Donation_Date.daysBetween(zoho.currentdate) >= 90;
}
```

C.2 Blood Request – DS-02 (On Success)

```
// DS-02 | Blood Request | Created -> Successful form submission
// Purpose: email every eligible donor whose blood group can donate to
the patient
if(input.Status == "Open")
{
    compat = Map();
    compat.put("O-ve",{"O-ve"});
    compat.put("O+ve",{"O-ve","O+ve"});
    compat.put("A-ve",{"O-ve","A-ve"});
    compat.put("A+ve",{"O-ve","O+ve","A-ve","A+ve"});
    compat.put("B-ve",{"O-ve","B-ve"});
    compat.put("B+ve",{"O-ve","O+ve","B-ve","B+ve"});
    compat.put("AB-ve",{"O-ve","A-ve","B-ve","AB-ve"});
    compat.put("AB+ve",{"O-ve","O+ve","A-ve","A+ve","B-ve","B+ve","AB-
ve","AB+ve"});
    groups = compat.get(input.Blood_Group_Needed);
    cutoff = zoho.currentdate.subDay(90);
    donors = Donor_Registration_Form[Last_Donation_Date == null ||
Last_Donation_Date <= cutoff];
    notified = 0;
    for each d in donors
    {
        if(groups.contains(d.Donor_Blood_Group1) && d.Donor_Email !=
null)
        {
            sendmail
            [
                from :zoho.adminuserid
                to :d.Donor_Email
                subject :input.Urgency + " blood request: " +
input.Blood_Group_Needed + " needed"
```

```
                message :"Dear " + d.Donor_Name.first_name +
",<br><br>A patient at " + input.Hospital + " needs " +
input.Units_Required + " unit(s) of " + input.Blood_Group_Needed + "
blood by " + input.Date_Required + ".<br>Your blood group (" +
d.Donor_Blood_Group1 + ") is compatible. Please reply if you can
donate.<br><br>Blood Donor Registry"
            ]
            notified = notified + 1;
        }
    }
    info "Donors notified: " + notified;
    sendmail
    [
        from :zoho.adminuserid
        to :zoho.adminuserid
        subject :"Request logged: " + input.Blood_Group_Needed + " ("
+ input.Urgency + ")"
        message :"A new blood request for " + input.Patient_Name + "
was logged. Eligible compatible donors notified: " + notified
    ]
}
```

APPENDIX D – USER MANUAL

D.1 Accessing the Application

Web: open https://creatorapp.zoho.in/adishwarseelan.sk.2024.cs215/blood-donor-registry-emergency-request-system/ and sign in with your Zoho account (invited by the administrator). Mobile: install the Zoho Creator app, sign in with the same account and open “Blood Donor Registry & Emergency Request System”.

D.2 Instructions for Blood Bank Coordinator

- To register a donor, open Donor Registrations → Donor Registration Form (or the Register Donor button on the dashboard), fill the mandatory fields and submit. Eligibility is set automatically.
- To log a request, open Blood Requests → Blood Request, enter patient details, blood group needed, units, required date, urgency and hospital, and submit. Compatible eligible donors are emailed automatically.
- Track requests in the Blood Requests Report (urgent first), the Request Pipeline Kanban or the Requirement Calendar; use the quick filters to narrow by blood group or status.
- When a donor has donated, record it in Donation Log (select the request and the donor, enter date and units), drag the request card to Fulfilled on the Request Pipeline, and update the donor’s Last date of donation.

D.3 Instructions for Administrator

- Invite users under Settings → User Permissions and assign the Write profile to coordinators and the Read profile to viewers.
- Make changes in the Development environment, test them, publish to Stage, re-test, then publish to Production.